SlugSync — Team Code Style Guide

This style guide documents the conventions the team follows so that code written by different members stays consistent and readable. It reflects the actual conventions used across the SlugSync codebase, verified by a review of the source. While we didn't inherently discuss some of these, by looking at how other members were writing their code and organizing it, the other members followed suite leading to a collective style throughout our project.

Language & Framework

- The frontend is written in React using functional components and hooks (useState, useEffect). Class components are not used anywhere.
- Files containing JSX use the .jsx extension; plain logic/helper files use .js.
- Supabase Edge Functions are written in TypeScript (Deno runtime).

Naming Conventions

- Components: PascalCase, matching the file name (e.g. EventForm.jsx exports EventForm). This is followed across all component and page files.
- Variables and functions: camelCase (e.g. handleSave, eventForm, fetchEvents). Note: snake_case appears only when mirroring Postgres column names (e.g. event_date, user_id), which is intentional and correct.
- Event handlers: complex or asynchronous handlers are prefixed with "handle" (e.g. handleSubmitEvent, handleAiParse, handleEventClick). Simple, synchronous UI handlers — toggles, open/close, and navigation — use plain descriptive verbs instead (e.g. openAiBox, closeForm, toggleInterest, goNext). This is the team's intentional convention: "handle" marks the more significant handlers.
- Boolean state variables read as yes/no questions where possible (e.g. isEditing, showForm, aiLoading).
- Constants that don't change use descriptive uppercase or camelCase names (e.g. HOURS, MINUTES, emptyForm).

File & Folder Organization

- Page-level components live in src/pages/ as .jsx files (e.g. Dashboard.jsx, Calendar.jsx, Profile.jsx).
- Reusable components live in src/components/ as .jsx files (e.g. EventForm.jsx).
- Data access and service/transform logic lives in src/data/ as .js files (e.g. eventService.js, formatEventRow.js, matchesPreferences.js).
- Context providers live in src/context/, custom hooks in src/hooks/, and Supabase client/utility setup in src/lib/.
- Supabase Edge Functions live in supabase/functions/ (e.g. parse-event, daily-digest, parse-source).

Component Structure

- State declarations (useState) are grouped near the top of a component.
- Helper and handler functions are defined within the component before the returned JSX.
- Small pure helper functions (e.g. date/time formatting) are defined outside the component, at the bottom of the file.
- Each component/page file has a single default export.

- Exception: context files (e.g. AuthContext.jsx, ThemeContext.jsx) intentionally use named exports — a Provider and a hook — rather than a single default export, since they are not component files in the usual sense.

Formatting

- Indentation is 2 spaces, applied consistently across all files (no tabs).
- Strings in JSX use double quotes. The one exception is where a string literal itself contains double quotes, in which case single quotes wrap it (e.g. a placeholder containing quoted example text).
- JSX attributes are written one concern per line when a tag has many props, for readability.

Data & Secrets

- API keys and secrets are never committed to the repository or placed in frontend code. No hardcoded keys exist in src/ or supabase/.
- The .env file is listed in .gitignore and is never pushed to GitHub. Only .env.example (with blank placeholder values) is tracked, to document which variables are needed.
- Environment variables exposed to the frontend are prefixed with VITE_ (e.g. VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY). The public Supabase key is safe to expose because Row-Level Security is the real authorization boundary.
- The one server-side secret, GEMINI_API_KEY, lives only in the Supabase Edge Function environment and is never shipped to the browser.

Version Control

- Work is done on feature branches named with the author and feature (e.g. jayna/calendar-feature), never directly on main.
- Commit messages clearly describe what changed.
- Changes are merged into main through Pull Requests, and merge conflicts are resolved before merging.
- node_modules and .env are gitignored; package-lock.json is committed to keep dependencies consistent across the team.

Unit Test Naming

- Test files are co-located with the file they test and named `[filename].test.js` or `[filename].test.jsx`, matching the extension of the file under test (e.g. `digestRange.js` → `digestRange.test.js`, `EventForm.jsx` → `EventForm.test.jsx`).
- Each test case describes the specific behavior being verified, stating the input condition and expected outcome.

Notes & Future Improvements

- No automated linter or formatter (ESLint/Prettier) is currently configured; style consistency has been maintained by team convention. Adding ESLint and Prettier is a candidate for future work to enforce these conventions automatically.

- A few filtering/transform helpers currently defined inline in Dashboard.jsx (e.g. sortEvents, buildMonthGrid) could be extracted into src/data/ for consistency with similar logic already located there.